

Vio 2 archivos

Vio 2 archivos

Perfecto. Tengo todo el contexto. Voy a hacer el análisis completo.

Ahora sí, con todo el contexto del stack de Debacu y los documentos. Voy a hacer el análisis completo.

Análisis preliminar de integración PMS para Debacu — lo que los documentos cubren muy bien, lo que falta, y cómo encajarlo con tu stack real.

Empiezo con la arquitectura por fases:



Y ahora el flujo de datos dentro de tu stack real (Edge Functions, Supabase, sin RPCs):



Ahora el análisis completo con todo lo que los documentos cubren bien, lo que falta, y las decisiones que tienes que tomar:

Lo que los documentos cubren muy bien

El modelo canónico universal está bien pensado: separación raw + canonical, multi-tenant por `org_id` + `property_id`, catálogos de mapping para estados/canales/segmentos, e idempotencia por (`connection_id`, `external_id`). El SQL es sólido y compatible con Supabase/Postgres. La estrategia de empezar read-only es correcta, reduce la fricción de aprobación con los PMS y el riesgo operativo.

Lo que falta o no está resuelto — y es crítico

1. GDPR y hashing de PII — el más urgente

Los documentos listan campos como `first_name`, `last_name`, `email`, `document_number` como campos planos en `pms_guests`. Pero tu arquitectura de Debacu ya establece que esos campos **nunca se guardan en claro** — van como `identity_key = HMAC(valor + pepper)`. Los documentos PMS no contemplan esto en absoluto. Hay que definir exactamente qué campos del guest se hashean,

cuáles se pueden guardar en claro (país, nacionalidad, `birth_date` probablemente), y cómo se hacen búsquedas sobre datos hashados (por ejemplo, detectar si un huésped ya existe).

2. RLS sobre las tablas PMS

El SQL del documento 4 no incluye ni una sola política RLS. En DebaCu todas las tablas tienen RLS habilitado. Hay que definir quién puede leer qué: ¿puede un usuario ver `pms_raw_records`? ¿Solo los Edge Functions con service role? ¿Los admins de la organización pueden ver los sync jobs de su propiedad?

3. Las credenciales PMS no tienen hogar definido

Los documentos dicen "almacenar cifradas, nunca en frontend" pero no dicen dónde concretamente en Supabase. Las opciones son Supabase Vault, una tabla separada con `SECURITY DEFINER` solo accesible desde Edge Functions, o un secrets manager externo. Hay que decidir esto antes de empezar a construir conectores.

4. El onboarding del hotel no está diseñado

Los documentos asumen que alguien configura la conexión manualmente. Pero si el objetivo es que sea "automático y fácil", necesitas un wizard de conexión que el propio hotel pueda completar: elegir PMS, introducir credenciales o hacer OAuth, lanzar el primer sync, ver el estado. Eso requiere una UI + Edge Functions de administración + gestión del estado de conexión — nada de esto está esbozado.

5. El orden de los PMS no está justificado operativamente

Los documentos proponen Tesipro/Ulyses como primer conector, pero Tesipro tiene API poco documentada públicamente. **Apaleo y Cloudbeds** son los mejores candidatos iniciales: API moderna, OAuth2, sandbox accesible, buena documentación, mercado compatible con hoteles medianos. Mews sería el tercero. Tesipro y Opera son para después.

6. No hay modelo de retry ni gestión de errores parciales

`pms_sync_jobs` tiene campos de estado, pero no hay lógica definida para reintentos: ¿qué pasa si un sync falla a mitad? ¿se reintentan desde el cursor o desde cero? ¿cuántas veces? ¿con backoff exponencial? Esto hay que definirlo en las Edge Functions de sincronización.

7. Los webhooks son opcionales en fase 1 pero necesitan endpoint desde el día 0

Aunque se empiece con polling, apaleo y Mews van a pedir declarar una webhook URL durante el proceso de aprobación. Ese endpoint tiene que existir aunque en fase 1 solo registre el evento y no lo procese. Si no existe, la aprobación se bloquea.

8. No hay estrategia de cuota y rate limiting contra los PMS

Cada PMS tiene límites de llamadas por minuto/hora. Si sincronizas 50 hoteles a la vez, puedes quemar el rate limit de un proveedor. Hay que diseñar un sistema de cola o scheduling que distribuya las sincronizaciones.

9. El pipeline ML no está conectado al modelo PMS

Los documentos PMS y los skills de ML pipeline están completamente separados. Hay que definir explícitamente qué tablas canónicas alimentan qué feature layer:

las `pms_reservations` normalizadas son la fuente de `lead_time`, `cancellation_ratio`, `booking_window`. Las `pms_stays` alimentan el forecasting de ocupación. Sin esta conexión definida, el valor de negocio de la integración no llega a los módulos.

Decisiones que tienes que tomar ahora

Antes de escribir una línea de código, necesitas responder a esto:

Primero, ¿cuál es el primer PMS? La recomendación clara es **apaleo** (OAuth2, API moderna, multi-property nativo, sandbox accesible) o **Cloudbeds** (muy extendido en hoteles medianos, API Key, más fácil de empezar). Elige uno.

Segundo, ¿dónde viven las credenciales PMS? Supabase Vault es la opción natural dentro de tu stack. Si no, una tabla `pms_credentials` con RLS estricto solo accesible desde Edge Functions con service role.

Tercero, ¿quién hace el onboarding — el hotel o tu equipo? Si es el hotel, necesitas el wizard de conexión desde fase 1. Si al principio lo hace tu equipo manualmente, puedes posponerlo a fase 2, pero tienes que decidirlo ya porque afecta qué construyes primero.

Próximo paso recomendado

El orden lógico dado tu stack y lo ya definido es: primero añadir RLS a todas las tablas PMS del SQL existente + resolver dónde van las credenciales. Luego construir una sola Edge Function de sync para apaleo que cubra el flujo completo (auth → fetch → raw → canonical → sync_job). Con eso funcionando en un hotel piloto real, tienes todo lo que necesitas para validar el modelo antes de escalar a más PMS o construir la UI de onboarding. ¿Quieres que empecemos por alguna de estas piezas en concreto?

EL PUNTO 3 HABRA QUE CREAR TABLA EN SUPABASE, PUNTO 4 LA IDEA ES QUE SI EL HOTEL CONTRATA PLAN CON API, EN MI CUENTA TENGA YA UN BOTON PARA CONECTRA LA API DEBEMOS DEJARLO TODO PREPARADO , SOLO SELECCIONAR DESDE UN DESPLEGABLE EL PMS QUE TIEE;NE Y EN ESE MOMENTO CONFIGURAR AUTOMATICAMENTE TODO (ES POSIBLE?) 5- ESTO HA SIDO PORQUE TENEMOS UN HOTEL (ORG) CON 3 PROPIEDADES (HOTELES) QUE NOS FACILITAN DATOS EN CSV Y USAN ESE PMS, PSIBLEMENTE NOS SEA MAS FACIL HABLAR CON ELLOS Y LOGRAR IMPLEMENTAR LA CONEXION DE DEBACU AL PMS, PERO AUN NO HAY NADA HASTA TENER ESTO DEFINIDO Y BIEN CONTROLADO. SI PIUNTO 8 LO DEJE PENDIENTE Y PUNTO 9 TAMBIEN HAY QUE PENSARLO Y DFINIRLO VBIEN PPREGUNTA PRIMERA EMPEZAMOS POR APALEO Y SI FUERA POSIBLE ULISES DE TESIPRO, SI NO APALEO PRIMERO. SEGUNDA PREGUNTA LA OPCION NATURAL O ES MAS SENCILLO PMS_CREDENTIALS CON rls Estricto. ANALIZALO Y ME DICES. PREGUNTA TERCERA AQUI TENGO DUDA LO SUYO ES QUE TENGAMOS UN BOTON Y QUE SEA EL HOTEL EL QUE QUIERA REALIAR LA CONSULTA RELLENANDO PARAMETROS

PUDIENDO HACER LA CONSULTA DE RIESGO DE CLIENTES O LA DE REVENUE (DEBACU SOLO LEERA DEL PMS) NO NECESITAMOS MAS EN ESTE MOMENTO EN EL FUTURO YA VEREMOS

Mostrar más

16 mar

Perfecto, ya tengo todo claro. Voy a analizar cada punto y darte respuestas concretas.

Respuesta 1 — Primer PMS: apaleo primero, Tesipro después

apaleo primero, sin duda. La razón es práctica: apaleo tiene OAuth2 estándar, API REST moderna, sandbox público, documentación excelente y proceso de partner razonable. Puedes empezar a construir hoy sin necesitar permiso de nadie.

Tesipro/Ulyses es diferente. No tiene API pública documentada de forma abierta — su integración depende de acuerdos directos con ellos. La ventaja que tienes es justo la que describes: ese hotel con 3 propiedades. Si consigues que ese cliente hable con Tesipro para facilitarte acceso, puedes construir el conector en paralelo o justo después de apaleo. Pero no construyas el conector Tesipro sin tener acceso real al sandbox primero — no hay suficiente documentación pública para hacerlo bien.

Orden real: apaleo → (en paralelo si hay acceso) Tesipro → Cloudbeds → Mews.

Respuesta 2 — Dónde guardar credenciales PMS

Te analizo las dos opciones dentro de tu stack real:

Supabase Vault es más seguro en teoría — los secretos viven cifrados fuera del schema público y Postgres nunca los expone en texto plano. Pero tiene un problema práctico importante: las Edge Functions acceden a Vault mediante `supabase.rpc('vault.decrypted_secrets')`, que en tu arquitectura es una RPC — y tu regla fundamental es **nunca usar RPCs**. Además Vault tiene limitaciones en el número de secretos y no está diseñado para escalar a cientos de conexiones PMS por organización.

Tabla `pms_credentials` con RLS estricto encaja perfectamente con tu stack. La implementación correcta es:

- Tabla separada del resto del modelo PMS
- Campos cifrados en la aplicación antes de insertar (cifrado en Edge Function, no en Postgres)
- RLS que solo permite acceso con `service_role` — ningún usuario autenticado puede leer esta tabla directamente, ni siquiera el admin del hotel
- Las Edge Functions de sync leen las credenciales con service role, las descifran en memoria, las usan para la llamada al PMS, y nunca las devuelven al frontend

La clave del cifrado vive en las variables de entorno de las Edge Functions, no en la base de datos.

La recomendación es `pms_credentials` con RLS estricto. Es más simple, más coherente con tu arquitectura, y escala bien.

La tabla quedaría así:

sql

```
create table pms_credentials (
  id uuid primary key default gen_random_uuid(),
  connection_id uuid not null references pms_connections(id) on delete cascade,
  org_id uuid not null,
  credential_type text not null, -- 'oauth_token' | 'api_key' |
'client_credentials'
  encrypted_value text not null, -- cifrado en Edge Function antes de insertar
  expires_at timestampz null, -- para tokens OAuth con expiración
  created_at timestampz not null default now(),
  updated_at timestampz not null default now(),
  unique (connection_id, credential_type)
);

-- RLS: solo service_role puede acceder
alter table pms_credentials enable row level security;
create policy "service_role_only" on pms_credentials
  using (false); -- bloquea todos los accesos vía JWT de usuario
```

Ningún usuario autenticado toca esta tabla. Solo las Edge Functions con service role.

Respuesta 3 — El wizard de conexión automática

Sí, es completamente posible y es exactamente el enfoque correcto. El flujo que describes — el hotel selecciona su PMS desde un desplegable y Debacu configura todo automáticamente — se puede construir limpio. Lo que necesitas preparar:

El flujo completo sería: hotel elige PMS en el desplegable → Debacu muestra el formulario de credenciales específico de ese PMS → hotel introduce sus datos (API key, o hace OAuth) → Edge Function valida la conexión, crea `pms_connection` y `pms_credentials`, lanza el primer sync → el hotel ve el estado en tiempo real y puede consultar riesgo o revenue desde ese momento.

Hay que diseñar bien tres cosas antes de construirlo.

Primera: cada PMS tiene un flujo de credenciales distinto. Para apaleo es OAuth2 — el hotel hace click en "Conectar con apaleo", se abre el OAuth flow de apaleo, vuelve con un token, Debacu lo guarda cifrado. Para Cloudbeds es API Key — el hotel la copia de su panel y la pega en Debacu. Para Tesipro probablemente sea un acuerdo manual al principio. El desplegable tiene que adaptar el formulario al PMS seleccionado.

Segunda: el wizard necesita validar antes de guardar. Antes de crear la conexión, una Edge Function tiene que hacer una llamada de prueba al PMS con las credenciales introducidas y confirmar que funcionan. Si falla, el hotel ve el error y puede corregirlo. Si funciona, se guarda todo y arranca el sync.

Tercera: el hotel solo ve el estado — nunca las credenciales. La pantalla post-conexión muestra: PMS conectado, último sync, número de reservas sincronizadas, estado. Un botón para relanzar

sync manual. Y los botones de consulta que mencionas: "Consultar riesgo de huéspedes" y "Consultar revenue" — que lanzan el análisis sobre los datos ya sincronizados.

Lo que Debacu hace en ese momento es solo leer del PMS y alimentar sus módulos. El hotel no necesita saber nada más.

Mapa completo de lo que hay que construir, en orden

Dado todo lo anterior, el orden de construcción real para llegar a ese botón funcionando es este:

Primero — SQL completo con RLS: tablas del modelo canónico + `pms_credentials` + políticas RLS en todas las tablas PMS. Sin esto nada es seguro.

Segundo — Edge Functions de conexión: crear conexión, validar credenciales contra el PMS, guardar cifrado, estado de conexión.

Tercero — Edge Function de sync para apaleo: full sync de room types, reservas y huéspedes. Con registro en `pms_sync_jobs` y persistencia raw + canonical.

Cuarto — UI del wizard: desplegable de PMS, formulario adaptativo, validación, estado de conexión, botones de consulta.

Quinto — Conectar con módulos de riesgo y revenue: las consultas que lanza el hotel leen de las tablas canónicas ya sincronizadas.

El hotel con Tesipro lo atacas en paralelo al tercero — si consigues acceso al sandbox o a su API a través de ese cliente, construyes el conector Tesipro sobre la misma base que apaleo. Si no, espera.

¿Empezamos por el SQL completo con RLS, o prefieres primero diseñar el wizard de UI para tener claro qué necesita el frontend antes de definir las Edge Functions?

GUARDA ESTE ANALISIS Y SEGUIREMOS MAÑANA

16 mar

Usó 4 herramientas

Usó 4 herramientas

Guardado. Mañana retomamos desde aquí — cuando quieras seguir solo dime y arrancamos directamente por donde lo dejamos. Que descanses.

podemos meter este chat en el poryecto debacu

21:33

Sí, puedes hacerlo fácilmente. Este chat está fuera de un proyecto ahora mismo. Para moverlo o incluirlo en el proyecto Debacu:

1. En el panel izquierdo de Claude.ai, busca el proyecto **Debacu**
2. Dentro del proyecto, hay una opción para **añadir conversaciones** o puedes simplemente **continuar la conversación desde dentro del proyecto**

Sin embargo, lo más práctico para que yo tenga contexto completo en futuras sesiones dentro del proyecto sería que **copies el contenido clave de este chat** (el análisis y las decisiones) y lo pegues

como un documento en los **archivos del proyecto** en DebaCu. Así cualquier conversación dentro del proyecto tendrá acceso directo a esas decisiones.

Lo que yo guardaría como documento en el proyecto sería exactamente el análisis de la sesión de hoy: las 6 fases, las decisiones sobre credenciales, el orden de construcción y los puntos pendientes.

¿Quieres que te lo prepare como un documento limpio listo para pegar en los archivos del proyecto?